

DESENVOLVIMENTO DE SISTEMAS WEB I

Aula 6: Create, Delete e o Caminho para o Banco de Dados

Fechando o CRUD em memória e conhecendo JDBC + PostgreSQL

Prática: Novo Contato e Excluir Contato · Teoria: JDBC, PostgreSQL e pgModeler

Objetivos da aula

1

Completar o CRUD em memória

Implementar Create (Novo Contato) e Delete (Excluir Contato).

2

Entender por que dados em memória não bastam

E o que muda quando migramos para um banco de verdade.

3

Conhecer o JDBC

A ponte entre o código Java e o banco de dados relacional.

4

Preparar o terreno para o PostgreSQL

Instalar a ferramenta de modelagem que usaremos: o pgModeler.

Agenda da aula

1 Recap: onde paramos no CRUD

2 Prática: Create — tela e lógica de Novo Contato

3 Prática: Delete — exclusão com confirmação

4 CRUD completo — revisão visual

5 Teoria: por que migrar para um banco de dados

6 Teoria: o que é JDBC

7 Teoria: por que PostgreSQL

8 Instalando o pgModeler para modelar o banco

Onde paramos no CRUD

C

Create

Criar

 **Hoje!**

R

Read

Ler / Listar

✓ **Feito**

U

Update

Atualizar

✓ **Feito**

D

Delete

Excluir

 **Hoje!**

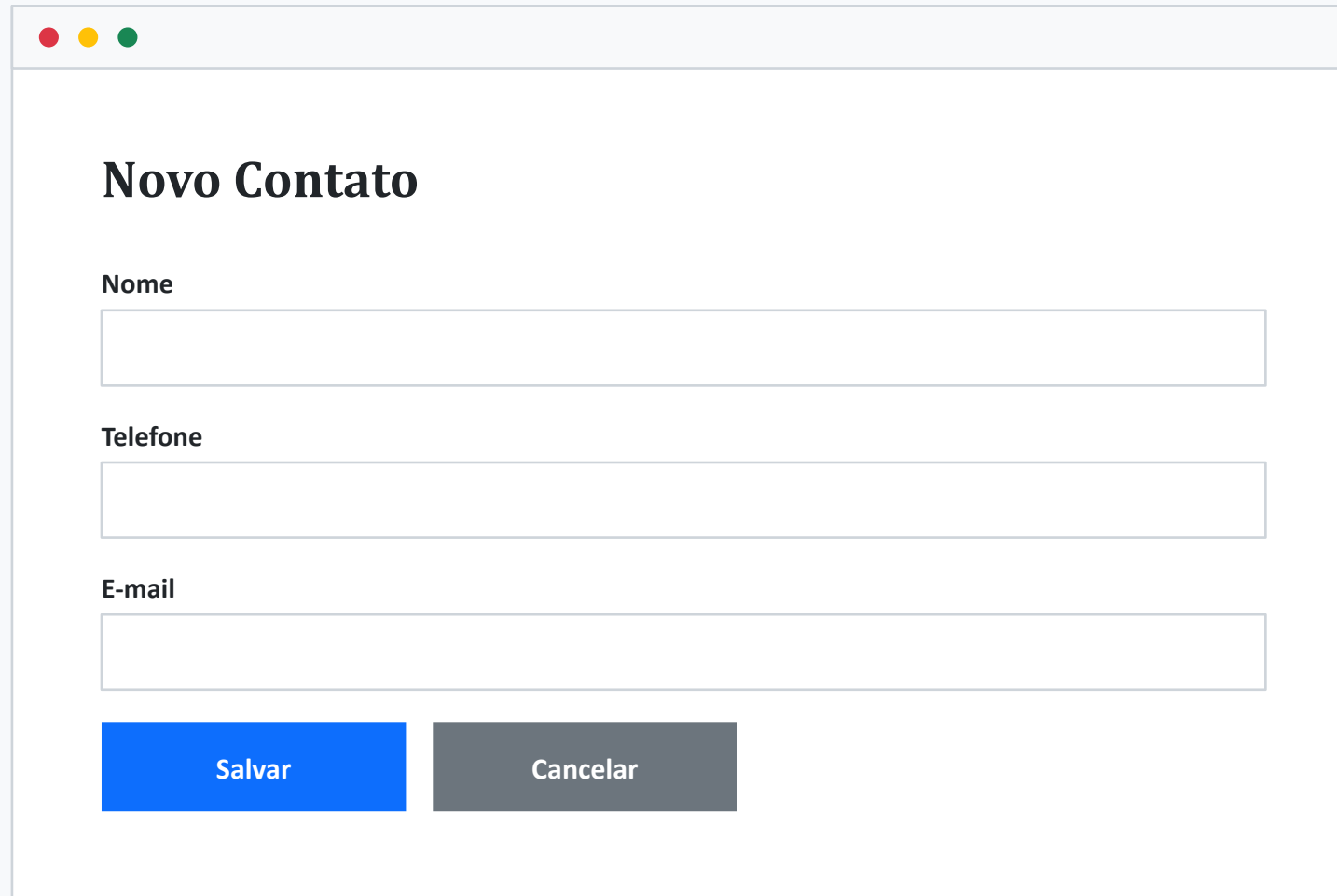
O ContatoDAO (Aula 5) já tem listar() e atualizar() — hoje adicionamos inserir() e excluir().

Requisitos da Tela Novo Contato

- 1 Formulário em branco (sem valores pré-preenchidos)
- 2 Campos: Nome, Telefone e E-mail
- 3 Sem campo de ID — quem decide o ID é o sistema, não o usuário
- 4 Botão "Salvar" cria o contato e volta para a Agenda
- 5 Botão "Cancelar" volta para a Agenda sem criar nada

Reaproveitamos quase tudo do editar-contato.jsp — a diferença é que o formulário nasce vazio.

Novo Contato



A screenshot of a web application window titled "Novo Contato". The window has a light gray title bar with three colored window control buttons (red, yellow, green) on the left. The main content area is white and contains the title "Novo Contato" in a bold, dark font. Below the title are three input fields, each with a label above it: "Nome", "Telefone", and "E-mail". All three input fields are empty. At the bottom of the form, there are two buttons: a blue "Salvar" button and a gray "Cancelar" button.

Novo Contato

Nome

Telefone

E-mail

Salvar Cancelar

Campos vazios — sem value nos inputs, ao contrário da tela de editar

Quem decide o próximo ID?

No formulário de criação não existe campo de ID — então quem gera esse número? Em memória, essa responsabilidade é do próprio ContatoDAO.

```
private static int proximoId = 3; // já existem os contatos 1 e 2

public static void inserir(Contato contato) {
    contato.setId(proximoId++);
    contatos.add(contato);
}
```

Isso é uma solução provisória. Quando migrarmos para o PostgreSQL, o próprio banco vai gerar esse número (coluna SERIAL / IDENTITY) — mais um motivo para não se apegar demais a essa lógica manual.

1

NovoContatoServlet

src/main/java/br/edu/faculdade/controller/NovoContatoServlet.java

```
@WebServlet("/novo-contato")
public class NovoContatoServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws Exception {
        request.getRequestDispatcher("novo-contato.jsp").forward(request, response);
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws Exception {
        Contato novo = new Contato(0,
            request.getParameter("nome"),
            request.getParameter("telefone"),
            request.getParameter("email"));

        ContatoDAO.inserir(novo);
        response.sendRedirect("agenda");
    }
}
```


Excluir contato — e por que não um simples link

Lembram do bloco de HTTP (Aula 1)? GET deveria ser idempotente — nunca causar um efeito colateral. Excluir É um efeito colateral. Por isso, exclusão vai por POST, não por um link comum.

✗ Jeito frágil

```
<a href="excluir?id=1">Excluir</a>
```

Um GET. Poderia ser disparado sem querer (pré-carregamento do navegador, um crawler, um F5) — e apagaria dados sem o usuário confirmar nada.

✓ Jeito correto

```
<form method="POST" action="excluir-contato">
```

Um formulário POST (mesmo que visualmente pareça um botão) + confirmação via JavaScript antes de enviar. Efeito colateral só acontece com ação explícita.

2

Implementando a exclusão

Coluna Opções em agenda.jsp:

```
<form action="excluir-contato" method="POST" style="display:inline"
    onsubmit="return confirm('Excluir este contato?');">
    <input type="hidden" name="id" value="${c.id}">
    <button type="submit" class="btn btn-danger btn-sm">Excluir</button>
</form>
```

ExcluirContatoServlet.java + ContatoDAO.java:

```
@WebServlet("/excluir-contato")
public class ExcluirContatoServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws Exception {
        int id = Integer.parseInt(request.getParameter("id"));
        ContatoDAO.excluir(id);
        response.sendRedirect("agenda");
    }
}

// dentro do ContatoDAO:
public static void excluir(int id) {
    contatos.removeIf(c -> c.getId() == id);
}
```

CRUD completo em memória!

C

R

U

D

Create, Read, Update e Delete — todos funcionando com o ContatoDAO em memória.

Mas o sistema tem um limite: se reiniciarmos o servidor agora, todos os dados voltam ao estado inicial. É hora de resolver isso de vez.

Por que uma lista em memória não basta

Não é persistente

Os dados vivem só enquanto o processo do Tomcat estiver rodando — um reinício apaga tudo.

Não é compartilhável

Se rodássemos duas instâncias da aplicação, cada uma teria sua própria lista, fora de sincronia.

Não escala

Uma List em memória não aguenta milhões de registros nem buscas complexas com boa performance.

Não é assim que sistemas reais funcionam

Todo sistema web sério guarda dados em um banco de dados — é hora de aprender a fazer isso de verdade.

O que é JDBC

JDBC (Java Database Connectivity) é a API padrão do Java para conectar e conversar com bancos de dados relacionais — é a "ponte" entre o seu código Java e o banco.

Driver

Uma biblioteca específica de cada banco (ex: postgresql-42.x.jar) que sabe "falar" com aquele SGBD.

Connection

Representa uma conexão aberta com o banco de dados.

PreparedStatement

Um comando SQL pronto para ser executado, com parâmetros seguros contra SQL Injection.

ResultSet

O resultado de uma consulta (SELECT) — percorrido linha a linha, como um cursor.

Como o ContatoDAO vai mudar

A boa notícia: quem CHAMA o DAO não vai mudar nada. Só o que está DENTRO dele.

HOJE

```
public static List<Contato> listar() {  
    return contatos;  
    // uma List em memória  
}
```

DEPOIS (com JDBC)

```
public static List<Contato> listar() {  
    // SELECT * FROM contato  
    // via Connection + PreparedStatement  
    // + ResultSet  
}
```

Isso só é possível porque, desde a Aula 3, os Servlets nunca acessam dados diretamente — sempre passam pelo DAO. Esse é o valor de programar em camadas.

Por que PostgreSQL

Open source e gratuito

Sem restrições de licença — inclusive as funcionalidades avançadas são todas livres.

Robusto e aderente ao padrão SQL

Ótimo para aprender SQL "de verdade", sem particularidades que só existem em um banco específico.

Muito usado no mercado

Um dos bancos relacionais mais adotados por empresas de todos os portes atualmente.

Versão estável atual: PostgreSQL 18

É essa a versão que vamos instalar e usar ao longo do restante da disciplina.

pgModeler — modelando o banco antes de codar

Antes de escrever qualquer SQL, vamos desenhar visualmente as tabelas, colunas e relacionamentos do nosso banco. O pgModeler é uma ferramenta open-source feita especificamente para modelar bancos PostgreSQL.

Diagrama ER visual

Cria tabelas e relacionamentos arrastando e conectando elementos na tela.

Gera o SQL automaticamente

O modelo visual vira código SQL (DDL) pronto para criar as tabelas no PostgreSQL.

Valida o modelo

Aponta erros e inconsistências antes de você rodar qualquer coisa no banco real.

Gratuito e multiplataforma

Community Edition open-source — Windows, Linux e macOS.

Preparando o ambiente de banco de dados

1

Instalar o PostgreSQL 18

postgresql.org/download — escolha seu sistema operacional. Durante a instalação, defina e ANOTE a senha do usuário "postgres".

2

Instalar o pgModeler

pgmodeler.io/download — versão 2.x, Community Edition (gratuita). Exige o PostgreSQL já instalado (ele usa o pg_dump/libpq por baixo dos panos).

3

Confirmar que os dois conversam

Abra o pgModeler e tente conectar ao PostgreSQL local — se a conexão funcionar, o ambiente está pronto.

Tragam os dois instalados na próxima aula — vamos modelar juntos, ao vivo, o banco de dados da Agenda Telefônica.

RESUMO

CRUD completo — e o mapa para o banco de verdade.

- 1 Hoje: Create e Delete implementados, CRUD 100% funcional em memória
- 2 Também hoje: base teórica de JDBC + escolha do PostgreSQL
- 3 Próxima aula: modelar o banco juntos com o pgModeler
- 4 Depois: trocar o ContatoDAO em memória por JDBC de verdade